

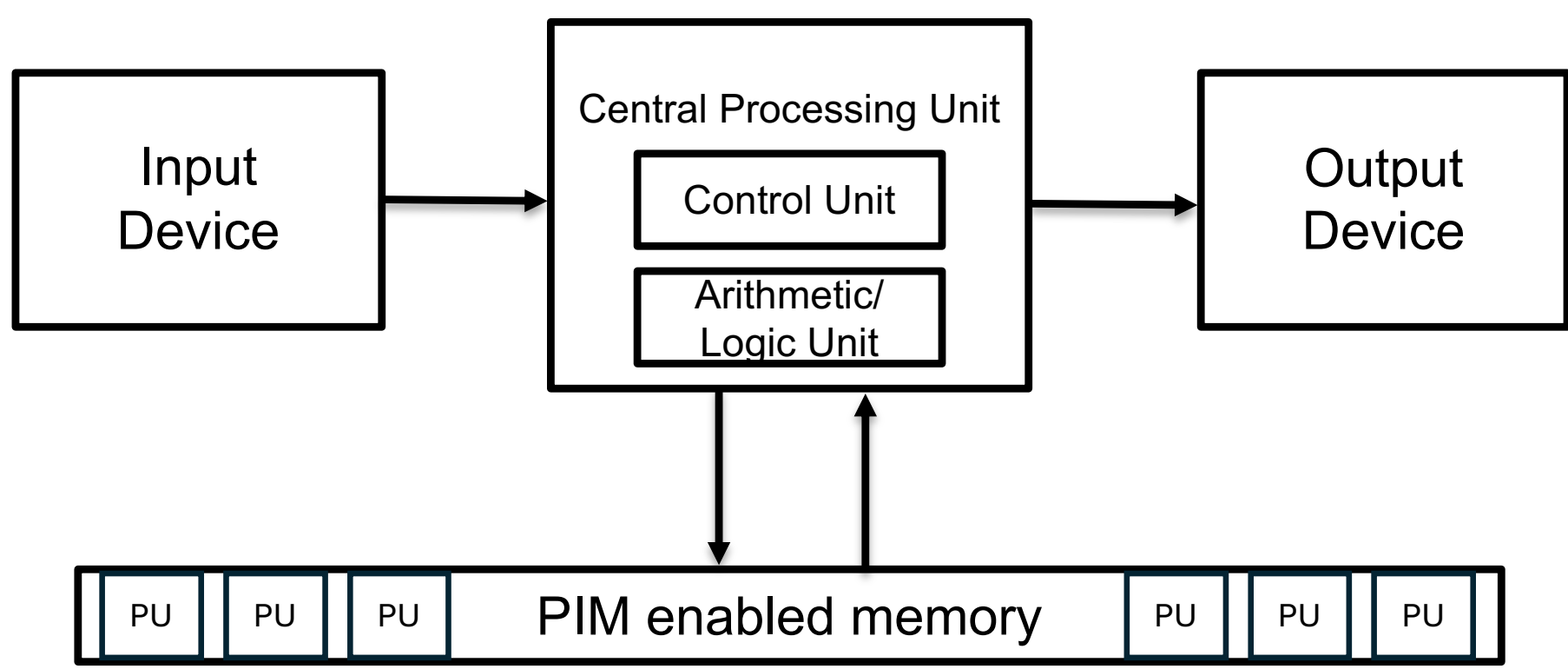
Introduction

- **The Problem:** Processing-in-Memory (PIM) reduces data movement but is notoriously hard to program.
- **Our Solution (PolymerPIM):** A native C++ or Julia framework for UPMEM that uses **deferred execution** to queue, fuse, and compile tasks at runtime.
- **Results:** Automatically generates optimized kernels via **vertical and horizontal fusion**, maximizing compute and **matching hand-tuned performance**.

Claim: Deferred execution unlocks kernel fusion, yielding highly efficient execution on UPMEM PIM systems.

Processing In Memory (PIM)

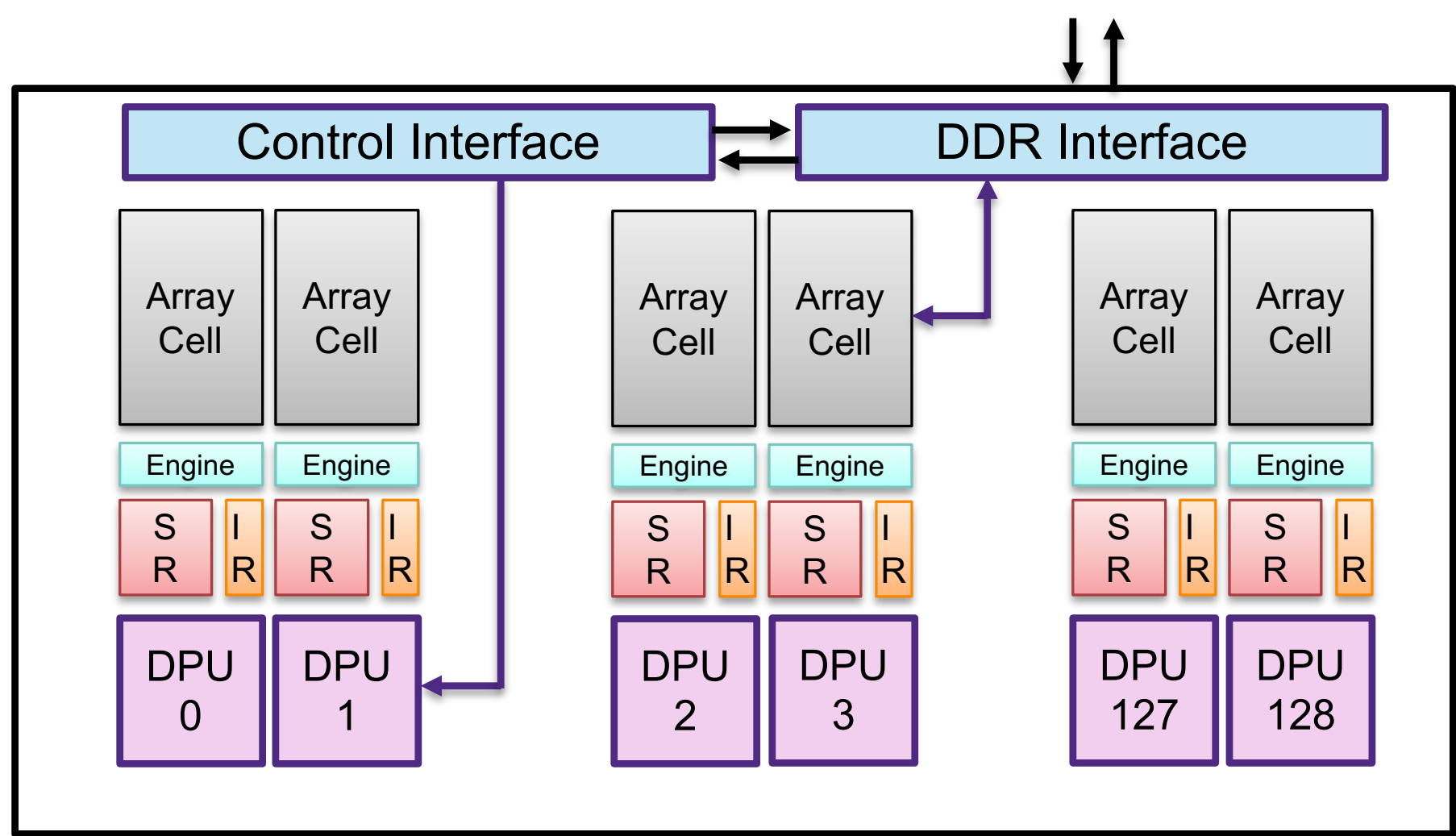
- Computation on DRAM to perform instructions
- Interrupting the typical compute centric architecture
- Reducing the data movement bottleneck



UPMEM PIM

UPMEM [2]: General purpose PIM architecture with a RISC ISA.

- DIMM replacement with 128 DRAM processing units (DPUs)
- 32bit processor with 24 threads and 14 pipeline stages
- Populate a single socket server with 2560 DPUs

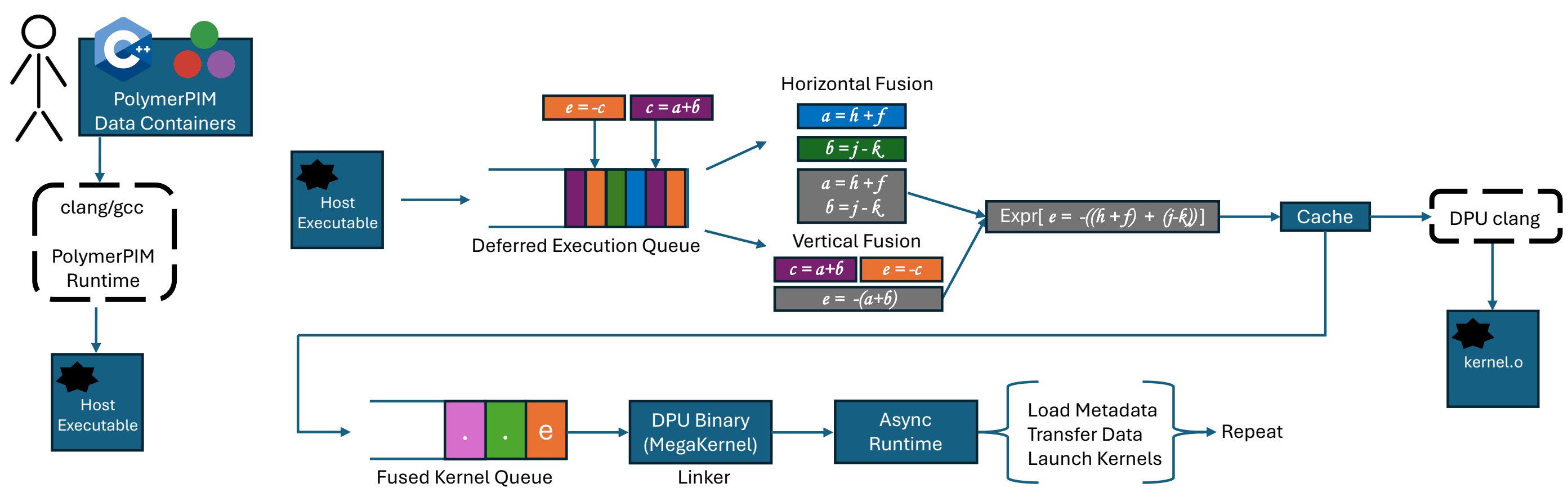


- Explicit kernel launches and memory transfers
- Compiler forked from LLVM12 providing a DPU target
- DPU device runtime is based on C99 standard

PolymerPIM

PolymerPIM, a deferred execution queue unlocks dependence-graph fusion to drive an asynchronous, task-based PIM runtime.

- **Frontend:** Abstract arrays (dpu_vector in C++, Base.Array in Julia).
- **Backend:** Asynchronous orchestration of transfers and execution.



JIT Design

PolymerPIM uses a deferred JIT compilation pipeline to eliminate mas-sive MRAM ↔ WRAM overheads:

- JIT-fuses operator sequences for maximum efficiency.
- Caches kernels into a single **megakernel**.
- Reduces kernel-switch overheads.



Figure 1. Scaling: Eager vs. Pipelined vs. JIT execution.

Graph Fusion Strategies

The deferred graph is optimized prior to compilation:

- **Vertical (Dependent):** Merges producer-consumer chains to bypass intermediate MRAM round-trips.
- **Horizontal (Independent):** Groups ready ops to saturate compute units and hide launch costs.

Hardware Constraints

DPU limits bound our optimizations:

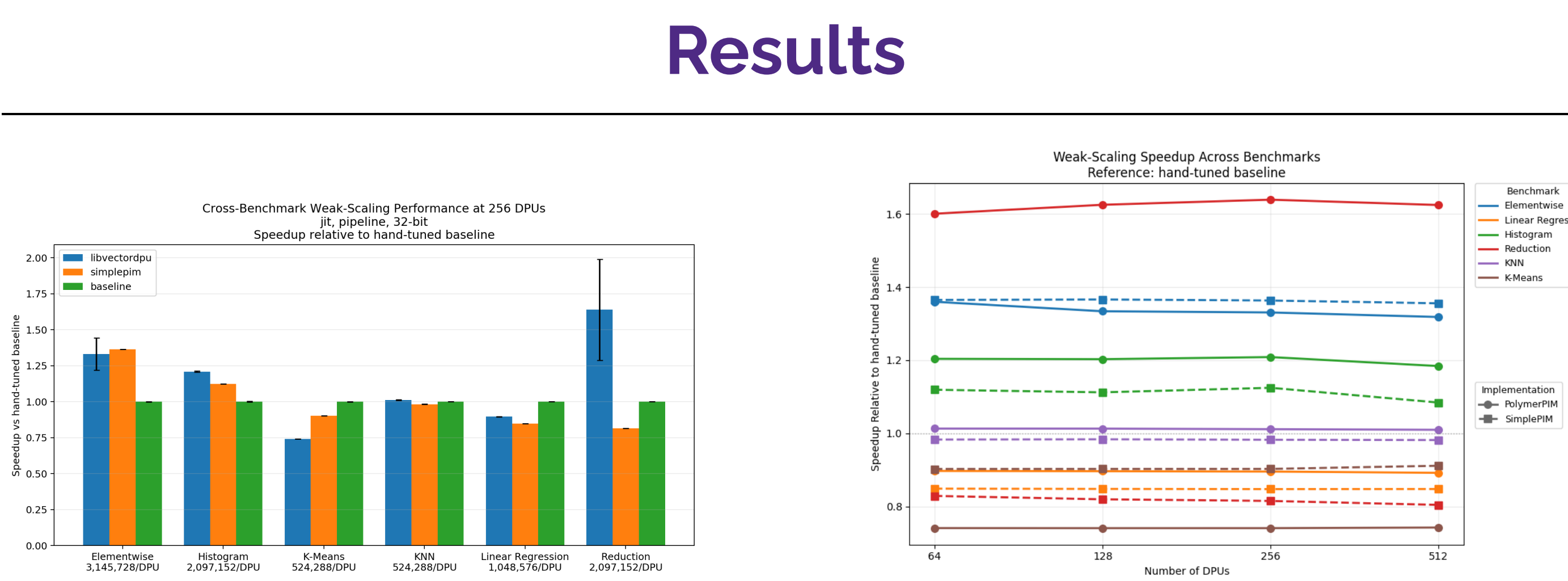
- **MRAM (64 MB):** Limits live vectors, which bottlenecks horizontal fusion.
- **WRAM (Scratchpad):** Exceeding capacity forces expensive multi-round data staging.

API Proposal

```
auto a = dpu_vector<int32_t>(n);
auto b = dpu_vector<int32_t>(n);
auto c = a + b;
```

	PolymerPIM	SimplePIM	Baseline
Elem	9	13	52
Hist	11	21	67
Red	7	21	50
Lin	25	54	73
KNN	32	57	99
K-Means	78	82	116
Total	162	248	457
% fewer	-	34.7%	64.6%

1. dpu_vector: Automated global DPU distribution and seamless arithmetic
2. dpu_local_vector: Explicit control over local DPU memory



These results are provided UPMEM server with 2560 DPUs across a wide diverse set of benchmarks.

PolymerPIM achieves **1.024x** geomean speedup over the best of SimplePIM [1] and the hand-tuned baseline.

PolymerPIM allows for efficient weak scaling across these benchmarks.

Conclusions

- **PolymerPIM:** Single-source C++/Julia model; reduces code size by 41–70%.
- **New Runtime:** First deferred, queue-based tasking for UPMEM.
- **Optimization:** Fuses tasks to cut communication and host overhead.

More details are found on our repo: github.com/krasow/PolymerPIM

Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, National Nuclear Security Administration, Office of Science, Office of Advanced Scientific Computing Research, Department of Energy Computational Science Graduate Fellowship under Award Number(s) DE-SC0024386.

[1] Jinfan Chen, Juan Gómez-Luna, Izzat El Hajj, Yuxin Guo, and Onur Mutlu. Simplepim: A software framework for productive and efficient processing-in-memory. In *Proceedings of the 32nd International Conference on Parallel Architectures and Compilation Techniques*, PACT '23, page 99–111. IEEE Press, 2024.

[2] UPMEM. <http://upmem.com>, 2024.